

Module 2

C Programming:

Translating pseudo code to C

Pre-Processor Directives

Input/Output

Operators

Structured Programming Concepts:

Selection (Simple if, if else, else if and switch statements)

Repetition statements(while, do while and for loop).

Introduction to Pseudocode

What is Pseudocode?

Pseudocode is a **simplified, half-English, half-code** representation of a computer algorithm. It is used for planning and designing programs before actual coding begins.

Purpose:

- Focuses on **algorithmic thinking** without worrying about syntax.
- Makes it easy to communicate logic to both programmers and non-programmers.
- Bridges the gap between human understanding and programming language.

Characteristics:

- Not language-specific
- Uses indentation and logical flow
- No strict syntax rules
- Easy to convert to any programming language

Example:

Start

Input a number

If number > 0

 Print "Positive"

Else

 Print "Non-positive"

End

Steps to Convert Pseudocode to C Program

Translating pseudocode into C involves systematic conversion of logic into code using C syntax.

Step-by-Step Guide:

Step	Description
1. Understand the Logic	Read and analyze the pseudocode carefully to understand its intent.
2. Identify Variables	Identify all variables used in the pseudocode and determine their data types (e.g., int, float).
3. Include Necessary Headers	Add <code>#include <stdio.h></code> for input/output operations.
4. Start the Main Function	Begin with <code>int main()</code> and enclose your code within <code>{ }</code> .
5. Declare Variables	Declare all identified variables at the beginning of <code>main()</code> .
6. Implement Input Statements	Use <code>scanf()</code> for each Input step in pseudocode.
7. Write Conditional Logic	Replace If, Else in pseudocode with <code>if</code> , <code>else</code> in C using proper syntax.
8. Use Output Statements	Replace Print with <code>printf()</code> statements.
9. Return from main()	Always include <code>return 0;</code> at the end of <code>main()</code> .
10. Compile and Test	Use a compiler (e.g., GCC) to run and test your program.

Example: Translating Pseudocode to C Program

◆ Pseudocode Example:

```
Start
Input a number
If number % 2 == 0
    Print "Even"
Else
    Print "Odd"
End
```

Equivalent C Program:

```
#include <stdio.h>

int main() {
    int number;

    // Input
    printf("Enter a number: ");
    scanf("%d", &number);

    // Decision making
    if (number % 2 == 0) {
        printf("Even\n");
    } else {
        printf("Odd\n");
    }

    return 0;
}
```

Explanation:

Pseudocode Concept C Code Translation

Start	int main() {
Input	scanf("%d", &number);
If...Else	if...else syntax
Print	printf()
End	return 0; }

Introduction to Preprocessor Directives in C

What are Preprocessor Directives?

Preprocessor directives in C are **instructions that are processed before the compilation of the program begins**. They tell the **C preprocessor** how to prepare the source code before it is compiled by the compiler.

Preprocessor directives begin with the **hash symbol (#)** and **do not end with a semicolon**.

Role of Preprocessor

The preprocessor performs tasks such as:

- Including header files
- Defining constants and macros
- Conditional compilation
- File inclusion
- Code optimization

Common Preprocessor Directives

Directive	Purpose	Example
#include	Inserts the contents of another file (usually a header)	#include <stdio.h>
#define	Defines symbolic constants or macros	#define PI 3.1415
#undef	Undefines a previously defined macro	#undef PI
#ifdef	Checks if a macro is defined	#ifdef DEBUG
#ifndef	Checks if a macro is not defined	#ifndef MAX
#endif	Ends a conditional directive block	#endif
#if, #else, #elif	Used for more complex conditional compilation	#if A > B
#pragma	Used for compiler-specific features	#pragma warning(disable:4996)

Categories of Preprocessor Directives

1. File Inclusion (#include)

- Includes standard or user-defined files.
- Two formats:
 - <filename> → for system header files
 - "filename" → for user-defined files

```
#include <stdio.h> // Standard header
#include "myfile.h" // User-defined file
```

2. Macro Substitution (#define)

- Substitutes values before compilation.

```
#define PI 3.14
#define SQUARE(x) ((x)*(x))
```

3. Conditional Compilation

- Compile code selectively based on conditions.

```
#define DEBUG

#ifdef DEBUG
    printf("Debug mode ON\n");
#endif
```

4. Other Directives

- #undef, #pragma, etc., for advanced control.

Example Code Using Preprocessor Directives

```
#include <stdio.h>
#define PI 3.1415
#define AREA(r) (PI * (r) * (r))

int main() {
    float radius = 5;
    float area = AREA(radius);
    printf("Area of circle: %.2f\n", area);
    return 0;
}
```

Introduction to Input/Output in C Programming

In C programming, **Input/Output (I/O)** refers to how a program communicates with the outside world—typically with the **user**.

- **Input** means receiving data from the user, most often via the **keyboard**.
- **Output** means displaying data or messages to the user, usually via the **monitor**.

C provides these capabilities through functions available in the **<stdio.h>** library (Standard Input/Output header file).

To use input and output functions in C, include this header:

```
#include <stdio.h>
```

Syntax and Types of Input/Output

C uses two primary functions for I/O:

Output – printf()

- Used to **print output** to the screen.
- Accepts a **format string** with placeholders (format specifiers) and corresponding variable values.

Syntax:

```
printf("format string", variable1, variable2, ...);
```

Example:

```
int age = 20;  
printf("Age is %d", age); // Output: Age is 20
```

Input – scanf()

- Used to **take input** from the user.
- Uses the **address-of operator (&)** to store user input into variables.

Syntax:

```
scanf("format string", &variable1, &variable2, ...);
```

Example:

```
int age;
```

```
printf("Enter your age: ");
scanf("%d", &age);
```

Common Format Specifiers

Data Type Format Specifier Example Input/Output

int	%d	42
float	%f	3.14
char	%c	'A'
string	%s	"Hello"
double	%lf	9.876

- ◆ \n is used for a **newline** in output.
- ◆ You **must use &** before variable names in scanf() except for strings.

Examples

Example 1: Integer Input and Output

```
#include <stdio.h>

int main() {
    int num;
    printf("Enter an integer: ");
    scanf("%d", &num);
    printf("You entered: %d", num);
    return 0;
}
```

Example 2: Input and Output of Multiple Data Types

```
#include <stdio.h>

int main() {
    int roll;
    float marks;
    char grade;

    printf("Enter roll number, marks, and grade: ");
    scanf("%d %f %c", &roll, &marks, &grade);
}
```

```
printf("Roll: %d\nMarks: %.2f\nGrade: %c", roll, marks, grade);  
return 0;  
}
```

Operators:

C supports a rich set of operators. An operator is a symbol that tells the computer to perform certain mathematical or logical manipulations. Operators are used in programs to manipulate data and variables. They usually form a part of the mathematical or logical expressions.

C operator can be classified into a number of categories. They include:

1. Arithmetic operators
2. Unary operator
3. Relational operators
4. Logical operators
5. Conditional operator
6. Bitwise operators
7. Increment and Decrement operators

ARITHMETIC OPERATORS

The basic operators for performing arithmetic are the same in many computer languages:

+	addition
-	subtraction
*	multiplication
/	division
%	modulus (remainder)

The - operator can be used in two ways: to subtract two numbers (as in $a - b$), or to negate one number (as in $-a + b$ or $a + -b$).

When applied to integers, the division operator / discards any remainder, so $1 / 2$ is 0 and $7 / 4$ is 1. But when either operand is a floating-point quantity (a real number), the division

operator yields a floating-point result, with a potentially nonzero fractional part. So $1 / 2.0$ is 0.5, and $7.0 / 4.0$ is 1.75.

The *modulus* operator `%` gives you the remainder when two integers are divided: $1 \% 2$ is 1; $7 \% 4$ is 3. (The modulus operator can only be applied to integers.)

An additional arithmetic operation you might be wondering about is exponentiation. Some languages have an exponentiation operator (typically `^` or `**`), but C doesn't. (To square or cube a number, just multiply it by itself.)

Multiplication, division, and modulus all have higher *precedence* than addition and subtraction. The term "precedence" refers to how "tightly" operators bind to their operands (that is, to the things they operate on). In mathematics, multiplication has higher precedence than addition, so $1 + 2 * 3$ is 7, not 9. In other words, $1 + 2 * 3$ is equivalent to $1 + (2 * 3)$. C is the same way.

All of these operators "group" from left to right, which means that when two or more of them have the same precedence and participate next to each other in an expression, the evaluation conceptually proceeds from left to right. For example, $1 - 2 - 3$ is equivalent to $(1 - 2) - 3$ and gives -4, not +2. ("Grouping" is sometimes called *associativity*, although the term is used somewhat differently in programming than it is in mathematics. Not all C operators group from left to right; a few groups from right to left.)

Whenever the default precedence or associativity doesn't give you the grouping you want, you can always use explicit parentheses. For example, if you want to add 1 to 2 and then multiply the result by 3, you could write $(1+2)*3$.

Program that shows the use of integer arithmetic to convert a given number of days into months and days.

```
/* Program to convert days to months and days */
main()
{
    int months, days;
    printf("Enter days\n");
    scanf("%d",&days);
```

```
months=days/30;
days=days%30;
printf("Months=%d      Days=%d", months,days);
}
```

UNARY OPERATOR

A unary operator acts upon a single operand to produce a new value.

Unary Minus

The most well known unary operator is minus, where a minus sign precedes a constant, variable or expression. In C, all numeric constants are positive. Therefore, a negative number is actually a positive constant preceded by a unary minus, for example:

-3.

RELATIONAL AND LOGICAL OPERATORS

An if statement like

```
if(x > max)
    max = x;
```

is perhaps deceptively simple. Conceptually, we say that it checks whether the condition $x > \text{max}$ is "true" or "false". The mechanics underlying C's conception of "true" and "false," however, deserve some explanation. We need to understand how true and false values are represented, and how they are interpreted by statements like if.

As far as C is concerned, a true/false condition can be represented as an integer. (An integer can represent many values; here we care about only two values: "true" and "false." The study of mathematics involving only two values is called Boolean algebra, after George Boole, a mathematician who refined this study.) In C, "false" is represented by a value of 0 (zero), and "true" is represented by any value that is nonzero. Since there are many nonzero values (at least 65,534, for values of type int), when we have to pick a specific value for "true," we'll pick 1.

The *relational operators* such as $<$, $<=$, $>$, and $>=$ are in fact operators, just like $+$, $-$, $*$, and $/$. The relational operators take two values, look at them, and "return" a value of 1 or 0

depending on whether the tested relation was true or false. The complete set of relational operators in C is:

<	less than
<=	less than or equal
>	greater than
>=	greater than or equal
==	equal
!=	not equal

For example, `1 < 2` is `true(1)`, `3 > 4` is `false(0)`, `5 == 5` is `true(1)`, and `6 != 6` is `false(0)`.

The equality-testing operator is `==`, not a single `=`, which is assignment. If you accidentally write

```
if(a = 0)
```

(and you probably will at some point; everybody makes this mistake), it will *not* test whether `a` is zero, as you probably intended. Instead, it will *assign* 0 to `a`, and then perform the “true” branch of the if statement if `a` is *nonzero*. But `a` will have just been assigned the value 0, so the “true” branch will never be taken! (This could drive you crazy while debugging -- you wanted to do something if `a` was 0, and after the test, `a` is 0, whether it was supposed to be or not, but the “true” branch is nevertheless not taken.)

The relational operators work with arbitrary numbers and generate true/false values. You can also combine true/false values by using the *Boolean operators* (also called the *logical operators*), which take true/false values as operands and compute new true/false values. The three Boolean operators are:

&&	AND
	OR
!	NOT (takes one operand, “unary”)

The `&&` (“and”) operator takes two true/false values and produces a true

(1) result if both operands are true (that is, if the left-hand side is true **and** the right-hand side is true). The `||` (“or”) operator takes two true/false values and produces a true (1) result

if either operand is true. The ! ("not") operator takes a single true/false value and negates it, turning false to true and true to false (0 to 1 and nonzero to 0). The logical operators && and || are used when we want to test more than one condition and make decisions.

For example, to test whether the variable *i* lies between 1 and 10, you might use

```
if(1 < i && i < 10)
```

...

Here we're expressing the relation "*i* is between 1 and 10" as "1 is less than *i* **and** *i* is less than 10."

It's important to understand why the more obvious expression

```
if(1 < i < 10) /* WRONG */
```

would not work. The expression $1 < i < 10$ is parsed by the compiler analogously to $1 + i + 10$. The expression $1 + i + 10$ is parsed as $(1 + i) + 10$ and means "add 1 to *i*, and then add the result to 10." Similarly, the expression $1 < i < 10$ is parsed as $(1 < i) < 10$ and means "see if 1 is less than *i*, and then see if the result is less than 10." But in this case, "the result" is 1 or 0, depending on whether *i* is greater than 1. Since both 0 and 1 are less than 10, the expression $1 < i < 10$ would *always* be true in C, regardless of the value of *i*!

Relational and Boolean expressions are usually used in contexts such as an if statement, where something is to be done or not done depending on some condition. In these cases what's actually checked is whether the expression representing the condition has a zero or nonzero value. As long as the expression is a relational or Boolean expression, the interpretation is just what we want. For example, when we wrote

```
if(x > max)
```

the > operator produced a 1 if *x* was greater than *max*, and a 0 otherwise.

The if statement interprets 0 as false and 1 (or any nonzero value) as true.

But what if the expression is not a relational or Boolean expression? As far as C is concerned, the controlling expression (of conditional statements like if) can in fact be *any* expression: it doesn't have to "look like" a Boolean expression; it doesn't have to contain relational or logical operators. All C looks at (when it's evaluating an if statement, or anywhere else where it needs a true/false value) is whether the expression evaluates to 0 or nonzero. For example, if you have a variable x, and you want to do something if x is nonzero, it's possible to write

```
if(x)
    statement
```

and the statement will be executed if x is nonzero (since nonzero means "true").

This possibility (that the controlling expression of an if statement doesn't have to "look like" a Boolean expression) is both useful and potentially confusing. It's useful when you have a variable or a function that is "conceptually Boolean," that is, one that you consider to hold a true or false (actually nonzero or zero) value. For example, if you have a variable verbose which contains a nonzero value when your program should run in verbose mode and zero when it should be quiet, you can write things like

```
if(verbose)
    printf("Starting first pass\n");
```

and this code is both legal and readable, besides which it does what you want. The standard library contains a function isupper() which tests whether a character is an upper-case letter, so if c is a character, you might write

```
if(isupper(c))
    ...
```

Both of these examples (verbose and isupper()) are useful and readable.

However, you will eventually come across code like

```
if(n)
    average = sum / n;
```

where n is just a number. Here, the programmer wants to compute the average only if n is nonzero (otherwise, of course, the code would divide by 0), and the code works, because, in

the context of the if statement, the trivial expression `n` is (as always) interpreted as "true" if it is nonzero, and "false" if it is zero.

"Coding shortcuts" like these can seem cryptic, but they're also quite common, so you'll need to be able to recognize them even if you don't choose to write them in your own code. Whenever you see code like

```
    if(x)
or
    if(f())
```

where `x` or `f()` do not have obvious "Boolean" names, you can read them as "if `x` is nonzero" or "if `f()` returns nonzero."

THE CONDITIONAL OPERATOR

The Conditional operator (ternary operator) pair `?:` is available in C to construct conditional expressions of the form *expr1?expr2:expr3*

where *expr1*, *expr2* and *expr3* are expressions.

The operator `?:` works as follows: *expr1* is evaluated first. If it is nonzero(true), then the expression *expr2* is evaluated and becomes the value of the expression. If *expr1* is false, *expr3* is evaluated and its value becomes the value of the expression. For example, consider the following statements:

```
a=100;
b=200;
c=(a>b)?a:b;
```

In this example, `c` will be assigned the value of `b`. This can be achieved using the `if..else` statements as follows: `if(a>b)`

```
c=a;
else
c=b;
```

The C language is accompanied by a number of *library functions* or *built in functions* that carry out various commonly used operations or calculations. There are library functions that carry

out standard input/output operations, functions that perform operations on characters, functions that perform operations on strings and functions that carry out various mathematical calculations.

Functionally similar library functions are usually grouped together as object programs in separate library files.

A library function is accessed simply by writing the function name, followed by a list of arguments that represent information being passed to the function. A function that returns a data item can appear anywhere within an expression in place of a constant or an identifier. A function that carries out operations on data items but does not return anything can be accessed simply by writing the function name.

A typical set of library functions will include a large number of functions that are common to most C compilers, such as those shown in table 2.1.

Function	Purpose
abs(i)	Return the absolute value of (i is integer)
ceil(d)	Round up to the next integer value(the smallest integer that is greater than or equal to d)
cos(d)	Return the cosine of d
exp(d)	Raise e to the power d(e=Naperian constant)
fabs(d)	Return the absolute value of d(d is double)
floor(d)	Round down to the next integer value(the largest integer that

	does not exceed d)
getchar()	Enter a character from the standard input device
log(d)	Return the natural logarithm of d
pow(d1,d2)	Return d1 raised to the power d2
putchar(c)	Send a character to the standard output device
rand()	Return a random positive integer
sin(d)	Return sine of d
sqrt(d)	Return the square root of d
tan(d)	Return the tangent of d
toascii(c)	Convert value of argument to ASCII
tolower(c)	Convert letter to lowercase
toupper(c)	Convert letter to uppercase

Program to convert lowercase to uppercase

```
#include <stdio.h> /* Input/Output functions are available in stdio.h
*/ #include<ctype.h> /* Character functions are available in the file
ctype.h */ main()
```



```

/* read a lowercase character and print its uppercase
equivalent */ {

    int lower, upper;
    lower=getchar();
    upper=toupper(lower);
    putchar(upper);
}

```

Program to illustrate the use of library functions

```

#include<stdio.h>
#include<ctype.h>
#include<math.h> /* Mathematical functions are available in math. h*/
main()
{
    int i=-10, e=2, d=10;
    float rad=1.57;
    double d1=2.0, d2=3.0;
    printf("%d\n", abs(i));
    printf("%f\n", sin(rad));
    printf("%f\n", cos(rad));
    printf("%f\n", exp(e));
    printf("%d\n", log(d));
    printf("%f\n", pow(d1,d2));
}

```

BITWISE OPERATORS

The *bitwise operators* &, |, ^, and ~ operate on integers thought of as binary numbers or strings of bits. The & operator is bitwise AND, the | operator is bitwise OR, the ^ operator is bitwise exclusive-OR (XOR), and the ~ operator is a bitwise negation or complement. (&, |, and ^ are "binary" in that they take two operands; ~ is unary.) These operators let you work with the individual bits of a variable; one common use is to treat an integer as a set of single-bit *flags*. You might define the 3rd bit as the "verbose" flag bit by defining

```
#define VERBOSE 4
```

Then you can “turn the verbose bit on” in an integer variable flags by executing

```
flags = flags | VERBOSE;
```

and turn it off with

```
flags = flags & ~VERBOSE;
```

and test whether it's set with

```
if(flags & VERBOSE)
```

The left-shift and right-shift operators << and >> let you shift an integer left or right by some number of bit positions; for example, value << 2 shifts value left by two bits.

The **comma operator** can be used to link the related expressions together. The expressions are executed one after the other. The most common use for comma operators is when you want multiple variables controlling a for loop, for example:

```
for(i = 0, j = 10; i < j; i++, j--)
```

INCREMENT AND DECREMENT OPERATORS

When we want to add or subtract constant 1 to a variable, C provides a set of shortcuts: the *autoincrement* and *autodecrement* operators. In their simplest forms, they look like this:

`++i` *add 1 to i*

`--j` *subtract 1 from j*

These correspond to the forms `i = i + 1` and `j = j - 1`. They are also equivalent to the short hand forms `i+=1` and `j-=1`. C has a set of ‘shorthand’ assignment operators of the form:

```
v op=exp;
```

where v is a variable, exp is an expression and op is a C binary arithmetic operator.

The assignment statement

```
v op=exp;
```

is equivalent to

```
v= v op(exp);
```

Example:

```
x+=y+1;
```

This is same as the statement

```
x=x+(y+1);
```

The ++ and -- operators apply to one operand (they're *unary* operators). The expression ++i adds 1 to i, and stores the incremented result back in i. This means that these operators don't just compute new values; they also modify the value of some variable. (They share this property -- modifying some variable -- with the assignment operators; we can say that these operators all have *side effects*. That is, they have some effect, on the side, other than just computing a new value.)

The incremented (or decremented) result is also made available to the rest of the expression, so an expression like

```
k = 2 * ++i
```

means "add one to i, store the result back in i, multiply it by 2, and store *that* result in k." (This is a pretty meaningless expression; our actual uses of ++ later will make more sense.)

Both the ++ and -- operators have an unusual property: they can be used in two ways, depending on whether they are written to the left or the right of the variable they're operating on. In either case, they increment or decrement the variable they're operating on; the difference concerns whether it's the old or the new value that's "returned" to the surrounding expression. The *prefix* form ++i increments i and returns the incremented value. The *postfix* form i++ increments i, but returns the *prior*, non-incremented value. Rewriting our previous example slightly, the expression

```
k = 2 * i++
```

means "take i's old value and multiply it by 2, increment i, store the result of the multiplication in k."

The distinction between the prefix and postfix forms of ++ and -- will probably seem strained at first, but it will make more sense once we begin using these operators in more realistic situations. For example,

```
a[i] = c;
```

```
i = i + 1;
```

using the ++ operator, we could simply write this

```
as a[i++] = c;
```

We wanted to increment *i* *after* deciding which element of the array to store into, so the postfix form *i*++ is appropriate.

Notice that it only makes sense to apply the ++ and -- operators to variables (or to other “containers,” such as *a[i]*). It would be meaningless to say something like

```
1++
```

or

```
(2+3)++
```

The ++ operator doesn't just mean “add one”; it means “add one *to a variable*” or “make a variable's value one more than it was before.” But (1+2) is not a variable, it's an expression; so there's no place for ++ to store the incremented result.

Another unfortunate example is

```
i = i++;
```

which some confused programmers sometimes write, presumably because they want to be extra sure that *i* is incremented by 1. But *i*++ all by itself is sufficient to increment *i* by 1; the extra (explicit) assignment to *i* is unnecessary and in fact counterproductive, meaningless, and incorrect. If you want to increment *i* (that is, add one to it, and store the result back in *i*), either use

```
i = i + 1;
```

or

```
i += 1;
```

or

```
++i;
```

or

```
i++;
```

Did it matter whether we used ++i or i++ in this last example? Remember, the difference between the two forms is what value (either the old or the new) is passed on to the surrounding expression. If there is no surrounding expression, if the ++i or i++ appears all by itself, to increment i and do nothing else, you can use either form; it makes no difference. (Two ways that an expression can appear “all by itself,” with “no surrounding expression,” are when it is an expression statement terminated by a semicolon, as above, or when it is one of the controlling expressions of a for loop.) For example, both the loops

```
for(i = 0; i < 10; ++i)
    printf("%d\n", i);
```

and

```
for(i = 0; i < 10; i++)
    printf("%d\n", i);
```

will behave exactly the same way and produce exactly the same results. (In real code, postfix increment is probably more common, though prefix definitely has its uses, too.)

THE SIZE OF OPERATOR

The **sizeof** is a compile time operator and, when used with an operand, it returns the number of bytes the operand occupies. The operand may be a variable, a constant or a data type qualifier.

Examples:

```
m=sizeof(sum);
n=sizeof(long int);
k=sizeof(235L);
```

The size of operator is normally used to determine the lengths of arrays and structures when their sizes are not known to the programmer. It is also used to allocate memory space dynamically to variables during execution of a program.

Program 2.4: Program to illustrate the use of size of operator

```
#include<stdio.h>
main()
```

```

{
    int i=10;
    printf("integer: %d\n", sizeof(i));
}

```

The above program might generate the following output:

integer: 2

Thus we see that this version of C allocates 2 bytes to each integer quantity.

Program 2.5: Program to illustrate arithmetic operators

```

#include<stdio.h>
main()
{
    int a, b, c, d;
    a=10;
    b=15;
    c=++a-b;
    printf("a=%d b=%d c=%d\n", a, b, c);
    d=b++ +a;
    printf("a=%d b=%d d=%d\n", a, b, d);
    printf("a/b=%d\n", a/b);
    printf("a%%b=%d\n", a%b);
    printf("a*=b=%d\n", a*=b);
    printf("%d\n", (c>d)?1:0);
    printf("%d\n", (c<d)?1:0);
}

```

PRECEDENCE OF OPERATORS

The precedence of C operators dictates the order of evaluation within an expression. The precedence of the operators introduced here is summarised in the Table 2.2. The highest precedence operators are given first.

Operators	Associativity

()->.	left to right
!~+-+--&*	right to left
* / %	left to right
+ -	left to right
<<=>>=	left to right
== !=	left to right
&	left to right
	left to right
&&	left to right
	right to left
=*= /= %= += -=	right to left

Where the same operator appears twice (for example *) the first one is the unary version.

A program to illustrate evaluation of expressions

```
#include<stdio.h>
main()
/* Evaluation of expressions */
{
    float a, b, c, x, y, z;
```

```

a=20;
b=2;
c=-23;
x = a + b / ( 3 + c * 4 - 1);
y = a - b / (3 + c) * ( 4 - 1);
z= a - ( b / ( 3 + c ) * 2 ) - 1;
printf( "x=%f\n", x);
printf("y=%f\n", y);
printf("z=%f\n", z);

}

```

Program to convert seconds to minutes and seconds #include <stdio.h>

```

#include <stdio.h>

#define SEC_PER_MIN 60    // seconds in a minute

int main(void)
{
    int sec, min, left;

    printf("Convert seconds to minutes and seconds!\n");
    printf("Enter the number of seconds you wish to
    convert.\n");
    scanf("%d", &sec);    *number of seconds is read in
    min = sec / SEC_PER_MIN; *truncated number of minutes
    left = sec % SEC_PER_MIN; *number of seconds left over
    printf("%d seconds is %d minutes, %d seconds.\n", sec,
    min,
        left);
    return 0;
}

```

Conditional statements.

INTRODUCTION

In the previous unit, you studied about the data types that are supported in C and the types of Input/output operators available in C. This unit will enable you to use the various types of

control statements for making a decision in C. Statements are the “steps” of a program. Most statements compute and assign values or call functions, but we will eventually meet several other kinds of statements as well. By default, statements are executed in sequence, one after another. We can, however, modify that sequence by using control flow constructs such that a statement or group of statements is executed only if some condition is true or false. This involves a kind of decision making to see whether a particular condition has occurred or not and then direct the computer to execute certain statements accordingly.

C language possesses such decision making capabilities and supports the following statements known as the control or decision making statements.

- **if** statement
- **switch** statement
- **goto** statement
- conditional operator statement

You will also study about the loops in this unit. Loops generally consist of two parts: one or more control expressions which control the execution of the loop, and the body, which is the statement or set of statements which is executed over and over.

As far as C is concerned, a true/false condition can be represented as an integer. (An integer can represent many values; here we care about only two values: “true” and “false.” The study of mathematics involving only two values is called Boolean algebra, after George Boole, a mathematician who refined this study.) In C, “false” is represented by a value of 0 (zero), and “true” is represented by any value that is nonzero. Since there are many nonzero values (at least 65,534, for values of type int), when we have to pick a specific value for “true,” we’ll pick 1.

Do...while loop is used in a situation where we need to execute the body of the loop before the test is performed. The for loop is used to execute the body of the loop for a specified number of times. The break statement is used to exit any loop.

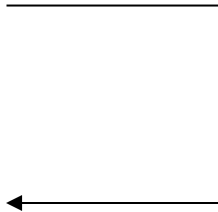
THE goto STATEMENT

C supports the **goto** statement to branch unconditionally from one point to another in the program. Although it may not be essential to use the **goto** statement in a highly structured language like C, there may be occasions when the use of **goto** might be desirable.

The **goto** requires a *label* in order to identify the place where the branch is to be made. A *label* is any valid variable name, and must be followed by a colon. The *label* is placed immediately before the statement where the control is to be transferred. The general forms of **goto** and *label* statements are shown below:

goto *label*;

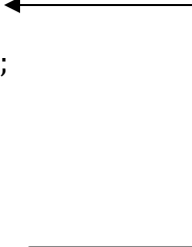
label:
statement;



(i) Forward jump

label:
statement;

goto *label*;



(ii) Backward jump

The *label* can be anywhere in the program either before the **goto** or after the **goto label**; statement.

During execution of the program when a statement like

goto first;

is met, the flow of control will jump to the statement immediately following the label **first**. This happens unconditionally.

Note that a **goto** breaks the normal sequential execution of the program. If the *label* is before the statement **goto label**; a loop will be formed and some statements will be executed repeatedly. Such a jump is known as a *backward jump*. On the other hand, if the *label* is placed after the **goto label**; some statements will be skipped and the jump is known as the *forward jump*.

A **goto** is often used at the end of a program to direct the control to go to the input statement, to read further data. Consider the following example:

Program showing unconditional branching

```
main()
{
    double a, b;
    read:
    printf("enter the value of a\n");
    scanf("%f", &a);
    if (a<0) goto read;
    b=sqrt(a);
    printf("%f %f \n",a, b);
    goto read;
}
```

This program is written to evaluate the square root of a series of numbers read from the terminal. The program uses two **goto** statements, one at the end, after printing the results to transfer the control back to the input statements and the other to skip any further computation when the number is negative.

Due to the unconditional **goto** statement at the end, the control is always transferred back to the input statement. In fact, this program puts the computer in a permanent loop known as an *infinite* loop.

THE IF STATEMENT

The simplest way to modify the control flow of a program is with an if statement, which in its simplest form looks like this:

```
if(x > max)
    max = x;
```

Even if you didn't know any C, it would probably be pretty obvious that what happens here is that if *x* is greater than *max*, *x* gets assigned to *max*. (We'd use code like this to keep track of the maximum value of *x* we'd seen--for each new *x*, we'd compare it to the old maximum value *max*, and if the new value was greater, we'd update *max*.)

More generally, we can say that the syntax of an if statement is:

```
if( expression )
    statement
```

where *expression* is any expression and *statement* is any statement.

What if you have a series of statements, all of which should be executed together or not at all depending on whether some condition is true? The answer is that you enclose them in braces:

```
if( expression )
{
    statement 1;
    statement 2;
    statement n;
}
```

As a general rule, anywhere the syntax of C calls for a statement, you may write a series of statements enclosed by braces. (You do not need to, and should not, put a semicolon after the closing brace, because the series of statements enclosed by braces is not itself a simple expression statement.)

Program to calculate the absolute value of an integer

```
#include <stdio.h>
void main ( )
{
    int number;
```

```

printf ("Type a
number:"); scanf ("%d",
& number);
if (number < 0) /* check whether the number is a negative number */
    number = - number; /* If it is negative then convert it into
    positive. */
printf ("The absolute value is % d \n", number);
}

```

The if-else statement

An **if** statement may also optionally contain a second statement, the “**else** clause,” which is to be executed if the condition is not met. Here is an example:

```

if(n > 0)
    average = sum / n;
else {
    printf("can't compute average\n");
    average = 0;
}

```

The first statement or block of statements is executed if the condition is true, and the second statement or block of statements (following the keyword **else**) is executed if the condition is *not true*. In this example, we can compute a meaningful average only if *n* is greater than 0; otherwise, we print a message saying that we cannot compute the average. The general syntax of an **if** statement is therefore

```

if( expression )
    statement(s)
else
    statement(s)

```

(if there are more than one statements, they should be enclosed within braces).

To find whether a number is negative or positive

```

#include < stdio.h >

```

```

void main ( )
{
    int num;
    printf ("Enter the number");
    scanf ("%d", &num);
    if (num < 0)
        printf ("The number is negative")
    else
        printf ("The number is positive");
}

```

Nesting of if statements

It's also possible to nest one **if** statement inside another. (For that matter, it's in general possible to nest any kind of statement or control flow construct within another.) For example, here is a little piece of code which decides roughly which quadrant of the compass you're walking into, based on an x value which is positive if you're walking east, and a y value which is positive if you're walking north:

```

if(x > 0)
{
    if(y > 0)
        printf("Northeast.\n");
    else    printf("Southeast.\n");
}

else {
    if(y > 0)
        printf("Northwest.\n");
    else    printf("Southwest.\n");
}

```

When you have one if statement (or loop) nested inside another, it's a very good idea to use explicit braces {}, as shown, to make it clear (both to you and to the compiler) how they're

nested and which **else** goes with which **if**. It's also a good idea to indent the various levels, also as shown, to make the code more readable to humans. Why do both? You use indentation to make the code visually more readable to yourself and other humans, but the compiler doesn't pay attention to the indentation (since all whitespace is essentially equivalent and is essentially ignored). Therefore, you also have to make sure that the punctuation is right.

Here is an example of another common arrangement of **if** and **else**. Suppose we have a variable **grade** containing a student's numeric grade, and we want to print out the corresponding letter grade. Here is the code that would do the job:

```
if(grade >= 90)
    printf("A");
else if(grade >= 80)
    printf("B");
else if(grade >= 70)
    printf("C");
else if(grade >= 60)
    printf("D");
else    printf("F");
```

What happens here is that exactly one of the five **printf** calls is executed, depending on which of the conditions is true. Each condition is tested in turn, and if one is true, the corresponding statement is executed, and the rest are skipped. If none of the conditions is true, we fall through to the last one, printing "F".

In the cascaded **if/else/if/else/...** chain, each **else** clause is another **if** statement. This may be more obvious at first **if** we reformat the example, including every set of braces and indenting each if statement relative to the previous one:

```
if(grade >= 90)
{
    printf("A");
}
```

```

else    {
        if(grade >= 80)
            {
                printf("B");
            }
        else    {
            if(grade >= 70)
                {
                    printf("C");
                }
            else    {
                if(grade >= 60)
                    {
                        printf("D");
                    }
                else    {
                    printf("F");
                }
            }
        }
    }
}

```

By examining the code this way, it should be obvious that exactly one of the **printf** calls is executed, and that whenever one of the conditions is found true, the remaining conditions do not need to be checked and none of the later statements within the chain will be executed. But once you've convinced yourself of this and learned to recognize the idiom, it's generally preferable to arrange the statements as in the first example, without trying to indent each successive **if** statement one tabstop further out.

Program to print the largest of three numbers

```

#include<stdio.h>
main()
{
    int a,b,c,big;
    printf ("Enter three numbers");

```



```

scanf ("%d %d %d", &a, &b, &c);
if (a>b) // check whether a is greater than b if true
    then if(a>c) // check whether a is greater than
        c
        big = a ; // assign a to big
    else big = c ; // assign c to big
else if (b>c) // if the condition (a>b) fails check whether b is greater
    than c big = b ; // assign b to big
else big = c ; // assign C to big
printf ("Largest of %d,%d&%d = %d", a,b,c,big);
}

```

THE CONDITIONAL EXPRESSION

The conditional operator (`?:`) takes three operands. It tests the result of the first operand and then evaluates one of the other two operands based on the result of the first. Consider the following example:

E1?E2:E3

If expression *E1* is nonzero (true), then *E2* is evaluated, and that is the value of the conditional expression. If *E1* is 0 (false), *E3* is evaluated, and that is the value of the conditional expression. Conditional expressions associate from right to left. In the following example, the conditional operator is used to get the minimum of *x* and *y*:

```
a = (x < y) ? x : y;    /* a= min(x, y) */
```

There is a sequence point after the first expression (*E1*). The following example's result is predictable, and is not subject to unplanned side effects:

```
i++ > j ? y[i] : x[i];
```

The conditional operator does not produce a lvalue. Therefore, a statement such as `a ? x : y = 10` is not valid.

THE SWITCH STATEMENT

The switch case statements are a substitute for long **if** statements that compare a variable to several "integral" values ("integral" values are simply values that can be expressed as an

integer, such as the value of a **char**). The basic format for using **switch case** is outlined below. The value of the variable given into **switch** is compared to the value following each of the cases, and when one value matches the value of the variable, the computer continues executing the program from that point. `switch ( <variable> ) {`

`case this-value:`

`Code to execute if <variable> == this-value`

`break;`

`case that-value:`

`Code to execute if <variable> == that-value`

`break;`

`...`

`default:`

`Code to execute if <variable> does not equal the value following any of the cases`

`break;`

`}`

The condition of a **switch** statement is a value. The **case** says that if it has the value of whatever is after that **case** then do whatever follows the colon. The **break** is used to **break** out of the case statements. **break** is a keyword that breaks out of the code block, usually surrounded by braces, which it is in. In this case, **break** prevents the program from falling through and executing the code in all the other **case** statements. An important thing to note about the **switch** statement is that the **case** values may only be constant integral expressions. It isn't legal to use **case** like this: `int a = 10;`

`int b = 10;`

`int c = 20;`

`switch ( a ) {`

`case b:`

`/* Code */`

`break;`

`case c:`

```
/* Code */  
break;  
default:  
/* Code */  
break;  
}
```

The **default** case is optional, but it is wise to include it as it handles any unexpected cases. It can be useful to put some kind of output to alert you to the code entering the **default** case if you don't expect it to. Switch statements serve as a simple way to write long if statements when the requirements are met. Often it can be used to process input from a user.

Example: Below is a sample program, in which not all of the proper functions are actually declared, but which shows how one would use switch in a program.

```
#include <stdio.h>  
void playgame();  
void loadgame();  
void playmultiplayer();  
int main()  
{  
    int input;  
    printf( "1. Play game\n" );  
    printf( "2. Load game\n" );  
    printf( "3. Play multiplayer\n" );  
    printf( "4. Exit\n" );  
    printf( "Selection: " );  
    scanf( "%d", &input );  
    switch ( input ) {  
        case 1: /* Note the colon, not a semicolon */ playgame();  
        break;  
        case 2:
```

```
loadgame();
break;
case 3:
playmultiplayer();
break;
case 4:
printf( "Thanks for playing!\n" );
break;
default:
printf( "Bad input, quitting!\n" );
break;
}
getchar();
}
```

This program will compile, but cannot be run until the undefined functions are given bodies, but it serves as a model (albeit simple) for processing input. If you do not understand this then try mentally putting in **if** statements for the **case** statements. **default** simply skips out of the **switch case** construction and allows the program to terminate naturally. If you do not like that, then you can make a loop around the whole thing to have it wait for valid input. You could easily make a few small functions if you wish to test the code.

THE WHILE LOOP

Loops generally consist of two parts: one or more *control expressions* which control the execution of the loop, and the body, which is the statement or set of statements which is executed over and over.

The most basic loop in C is the **while** loop. A **while** loop has one control expression, and executes as long as that expression is true. Here before executing the body of the loop, the condition is tested. Therefore it is called an **entry-controlled** loop. The following example repeatedly doubles the number 2 (2, 4, 8, 16, ...) and prints the resulting numbers as long as they are less than 1000:

```
int x = 2;

while(x < 1000)
{
    printf("%d\n", x);
    x = x * 2;
}
```

(Once again, we've used braces `{ }` to enclose the group of statements which are to be executed together as the body of the loop.)

The general syntax of a **while** loop is

```
while( expression )
    statement(s)
```

A **while** loop starts out like an **if** statement: if the condition expressed by the expression is true, the *statement* is executed. However, after executing the statement, the condition is tested again, and if it's still true, the statement is executed again. (Presumably, the condition depends on some value which is changed in the body of the loop.) As long as the condition remains true, the body of the loop is executed over and over again. (If the condition is false right at the start, the body of the loop is not executed at all.)

As another example, if you wanted to print a number of blank lines, with the variable **n** holding the number of blank lines to be printed, you might use code like this:

```
while(n > 0)
{
    printf("\n");
    n = n - 1;
}
```

After the loop finishes (when control "falls out" of it, due to the condition being false), **n** will have the value 0.

You use a **while** loop when you have a statement or group of statements which may have to be executed a number of times to complete their task. The controlling expression represents the condition "the loop is not done" or "there's more work to do." As long as the expression

is true, the body of the loop is executed; presumably, it makes at least some progress at its task. When the expression becomes false, the task is done, and the rest of the program (beyond the loop) can proceed. When we think about a loop in this way, we can see an additional important property: if the expression evaluates to "false" before the very first trip through the loop, we make *zero* trips through the loop. In other words, if the task is already done (if there's no work to do) the body of the loop is not executed at all. (It's always a good idea to think about the "boundary conditions" in a piece of code, and to make sure that the code will work correctly when there is no work to do, or when there is a trivial task to do, such as sorting an array of one number. Experience has shown that bugs at boundary conditions are quite common.)

Program to find largest of n numbers

```
main()
{
    int num, large, n, i;
    clrscr();
    printf("enter number of numbers \n");
    scanf("%d",&n);
    large=0;

    i=0;
    while(i<n)
    {
        printf("\n enter number ");
        scanf("%d", &num);
        if(large<num)
            large=num;
        i++;
    }
    printf("\n large = %d", large);
}
```

Program to evaluate sine series $\sin(x)=x-x^3/3!+x^5/5!-x^7/7!+-----$ depending on accuracy

```

#include<stdio.h>
#include
<math.h> void
main()
{
    int n, i=1, count;
    float acc, x, term, sum;
    printf("enter the angle\n");
    scanf("%d", &x);
    x=x*3.1416/180.0;
    printf("\nenter the
    accuracy"); scanf("%f",
    &acc);
    sum=x;
    term=x;
    while ((fabs(term))>acc)
    {
        term=-
        term*x*x/((2*i)*(2*i+1));
        sum+=term;
        i++;
    }
    printf("\nsum of sine series is %f", sum);
}

```

THE DO...WHILE LOOP

The **do...while** loop is used in a situation where we need to execute the body of the loop before the test is performed. Therefore, the body of the loop may not be executed at all if the condition is not satisfied at the very first attempt. Where as while loop makes a test of condition before the body of the loop is executed.

For above reasons **while** loop is called an **entry-controlled loop** and **do..while** loop is called an **exit-controlled loop**.

do while loop takes the following form:

```

do
{
    Body of the loop
}
while ( expression);

```

On reaching the **do** statement , the program proceeds to evaluate the body of the loop first. At the end of the loop, the conditional expression in the **while** statement is evaluated. If the expression is true, the program continues to evaluate the body of the loop once again. This process continues as long as the expression is true. When the expression becomes false, the loop will be terminated and the control goes to the statement that appears immediately after the **while** statement.

On using the do loop, the body of the loop is always executed at least once irrespective of the expression.

A program to print the multiplication table from 1 x 1 to 10 x 10 as shown below using do-while loop.

1	2	3	4		10
2	4	6	8		20
3	6	9	12		30
4					40
.					.
.					.
10					100

```

// Program to print
multiplication table main()
{
    int
    rowmax=10,colmax=10,row,col,x;
    printf(" Multiplication table\n");
    printf(" ..... \n");
    row=1;

```



```

do
{
    col=1
; do
{
    x=row*col;
    printf("%4d",
    x); col=col+1;
}
while
(col<=colmax);
printf("\n");
row=row+1;
}
while(row<=rowmax);
Printf(" ..... \
n");
}

```

THE FOR LOOP

The **for** loop is used to repeat the execution of set of statements for a fixed number of times.

The **for** loop is also an **entry-controlled loop**.

Generally, the syntax of a for loop is

```
for(expr1; expr2; expr3) statement(s)
```

(Here we see that the **for** loop has three control expressions. As always, the statement can be a brace-enclosed block.)

Many loops are set up to cause some variable to step through a range of values, or, more generally, to set up an initial condition and then modify some value to perform each succeeding loop as long as some condition is true. The three expressions in a **for** loop encapsulate these conditions: expr1 sets up the initial condition, expr 2 tests whether another trip through the loop should be taken, and expr3 increments or updates things after each trip through the loop and prior to the next one. Consider the following :

```
for (i = 0; i < 10; i = i + 1)
```

```
printf("i is %d\n", i);
```

In the above example, we had $i = 0$ as `expr1`, $i < 10$ as `expr2`, $i = i + 1$ as `expr3`, and the call to **printf** as statement, the body of the loop. So the loop began by setting i to 0, proceeded as long as i was less than 10, printed out i 's value during each trip through the loop, and added 1 to i between each trip through the loop.

When the compiler sees a for loop, first, `expr1` is evaluated. Then, `expr2` is evaluated, and if it is true, the body of the loop (statement) is executed. Then, `expr3` is evaluated to go to the next step, and `expr2` is evaluated again, to see if there is a next step. During the execution of a for loop, the sequence is:

```
expr1
expr2
statement
expr3
expr2
statement
expr3
...
expr2
statement
expr3
expr2
```

The first thing executed is `expr1`. `expr3` is evaluated after every trip through the loop. The last thing executed is always `expr2`, because when `expr2` evaluates false, the loop exits.

All three expressions of a for loop are optional. If you leave out `expr1`, there simply is no initialization step, and the variable(s) used with the loop had better have been initialized already. If you leave out `expr2`, there is no test, and the default for the for loop is that another trip through the loop should be taken (such that unless you break out of it some other way, the loop runs forever). If you leave out `expr3`, there is no increment step.

The semicolons separate the three controlling expressions of a for loop. (These semicolons, by the way, have nothing to do with statement terminators.) If you leave out one or more of the expressions, the semicolons remain. Therefore, one way of writing a deliberately infinite loop in C is

```
for(;;)
```

```
...
```

It's also worth noting that a for loop can be used in more general ways than the simple, iterative examples we've seen so far. The "control variable" of a for loop does not have to be an integer, and it does not have to be incremented by an additive increment. It could be "incremented" by a multiplicative factor (1, 2, 4, 8, ...) if that was what you needed, or it could be a floating-point variable, or it could be another type of variable which we haven't met yet which would step, not over numeric values, but over the elements of an array or other data structure. Strictly speaking, a **for loop** doesn't have to have a "control variable" at all; the three expressions can be anything, although the loop will make the most sense if they are related and together form the expected initialize, test, increment sequence.

The powers-of-two example using **for** is:

```
int x;
for(x = 2; x < 1000; x = x * 2)
    printf("%d\n", x);
```

There is no earth-shaking or fundamental difference between the **while** and **for loops**. In fact, given the general for loop

```
for(expr1; expr2; expr3)
    statement
```

you could usually rewrite it as a while loop, moving the initialize and increment expressions to statements before and within the loop:

```
expr1;
while(expr2)
{
    statement
```

```
    expr3;  
}
```

Similarly, given the general while loop

```
while(expr)  
    statement
```

you could rewrite it as a for loop:

```
for(; expr; )  
    statement
```

Another contrast between the **for** and **while** loops is that although the test expression (expr2) is optional in a **for** loop, it is required in a **while** loop. If you leave out the controlling expression of a **while** loop, the compiler will complain about a syntax error. (To write a deliberately infinite **while** loop, you have to supply an expression which is always nonzero. The most obvious one would simply be while(1) .)

If it's possible to rewrite a **for** loop as a **while** loop and vice versa, why do they both exist? Which one should you choose? In general, when you choose a for loop, its three expressions should all manipulate the same variable or data structure, using the initialize, test, increment pattern. If they don't manipulate the same variable or don't follow that pattern, wedging them into a **for** loop buys nothing and a **while** loop would probably be clearer. (The reason that one loop or the other can be clearer is simply that, when you see a **for** loop, you expect to see an idiomatic initialize/ test/increment of a single variable, and if the **for** loop you're looking at doesn't end up matching that pattern, you've been momentarily misled.)

A Program to find the factorial of a number

```
void main()  
{  
    int M,N;  
    long int F=1;  
    clrscr();  
    printf("enter the number\n");  
    scanf("%d",&N);
```

```

if(N<=0)
F=1;
else
{
for(M=1;M<=N;M++)
F*=M;
}
printf("the factorial of the number is %ld",F);
getch();
}

```

The Nesting of for loops

Nesting of for loops, that is, one for statement within another for statement, is allowed in C. For example, two loops can be nested as follows:

```

.....
.....
for(i=1;i<10;i++)
{
.....
.....
for(j=1;j<5;j++)
{
.....
.....
}
.....
.....
}
.....
.....

```

THE BREAK STATEMENT AND CONTINUE STATEMENT

The purpose of **break** statement is to break out of a loop (**while**, **do while**, or **for** loop) or a **switch** statement. When a **break** statement is encountered inside a loop, the loop is

immediately exited and the program continues with the statement immediately following the loop. When the loops are nested, the **break** would only exit from the loop containing it. That is, the **break** would exit only a single loop.

Syntax : break;

Program to illustrate the use of break statement.

```
void main ( )
{
    int x;
    for (x=1; x<=10; x++)
    {
        if (x==5)
            break; /*break loop only if x==5 */
        printf("%d", x);
    }
    printf ("\nBroke out of loop");
    printf( "at x =%d");
}
```

The above program displays the numbers from 1 to 4 and prints the message "Broke out of loop when 5 is encountered."

The continue statement

The **continue** statement is used to continue the next iteration of the loop by skipping a part of the body of the loop (for, **do/while** or **while** loops). The continue statement does not apply to a **switch**, like a **break** statement.

Unlike the **break** which causes the loop to be terminated, the **continue**, causes the loop to be continued with the next iteration after skipping any statements in between.

Syntax: continue;

Program to illustrate the use of continue statement.

```
void main ( ) {
```

```

int x;
for (x=1; x<=10; x++)
    { if (x==5)
        continue; /* skip remaining code in loop
                    only if x == 5 */
        printf ("%d\n", x);
    }
printf("\nUsed continue to skip");
}

```

The above program displays the numbers from 1 to 10, except the number 5.

Program to sum integers entered interactively

```

#include <stdio.h>
int main(void)
{
    long num;
    long sum = 0L;    /* initialize sum to zero */
    int status;

    printf("Please enter an integer to be summed.
"); printf("Enter q to quit.\n"); status =
scanf("%ld", &num);
    while (status == 1)
    {
        sum = sum + num;
        printf("Please enter next integer to be
summed. "); printf("Enter q to quit.\n"); status
= scanf("%ld", &num);
    }
    printf("Those integers sum to %ld.\n", sum);
    return 0;
}

```